

Bản sơ về ứng dụng hàm đánh giá trong thi tin học

Zhou Erjin

Trường Trung học số 1 Thiệu Hưng, Chiết Giang

2009

Nguồn gốc: Bản sơ về ứng dụng hàm đánh giá trong thi tin học

IOI China candidate team papers / 2009 / Zhou Erjin / presentation slides

Abstract

Nguồn là bộ slide trình bày đi kèm bài viết về hàm đánh giá. Bản tiếng Việt này chuyển nội dung slide thành một bài ghi chú có cấu trúc, giữ các ví dụ tìm kiếm, tối ưu quy hoạch động, lời giải gần tối ưu và các bảng số liệu cốt lõi; những slide lặp lại đề mục hoặc hiệu ứng trình chiếu được gộp vào phần văn bản tương ứng.

Mở đầu

Trong các bài toán có không gian trạng thái rất lớn và chuyển trạng thái tốn thời gian, hàm đánh giá có vai trò quan trọng. Nó ánh xạ mức tốt xấu của một trạng thái trong phần giải tiếp theo về một giá trị cụ thể, nhờ đó chương trình có thể quyết định trạng thái nào nên được giữ lại, cắt bỏ hoặc ưu tiên.

Các ghi chú trình bày ba hướng dùng hàm đánh giá:

1. trong tìm kiếm, gồm bài toán tối ưu và bài toán khả thi;
2. trong tối ưu quy hoạch động;
3. trong việc tìm lời giải khá tốt khi nghiệm tối ưu quá khó đạt được.

1 Hàm đánh giá trong tìm kiếm

1.1 Ứng dụng trong một lớp bài toán tối ưu

Uva 10605 – Mines For Diamonds

Cho một khu mỏ kim cương kích thước $n \times m$. Ta cần đào một số đường hầm để lấy hết kim cương. Một đường hầm là một đường gấp khúc xuất phát từ biên khu mỏ; mọi viên kim cương nằm trên đường đó đều được lấy. Hai đường gấp khúc không được cắt nhau, nhưng có thể kề nhau. Hãy tìm tổng độ dài đường hầm nhỏ nhất. Dữ liệu bảo đảm $n, m \leq 11$, số kim cương $k \leq 10$.

Thoạt nhìn, n, m nhỏ và số kim cương cũng ít, nên tìm kiếm vét cạn có vẻ tự nhiên. Tuy nhiên số nút sinh ra tăng rất nhanh. Một bộ dữ liệu minh họa trong bài trình chiếu cho bảng sau:

Độ sâu	Số nút	Độ sâu	Số nút
1	9	10	702437
2	41	11	2191682
3	145	12	6705638
4	515	13	20066463
5	1776	14	58721500
6	6120	15	167773977
7	20511	16	467536220
8	67935	17	120043037
9	220273		

Vì vậy tối ưu là bắt buộc. Gọi n là một trạng thái bất kỳ trong tìm kiếm, $g(n)$ là chi phí từ trạng thái đầu tới n , $h(n)$ là ước lượng chi phí từ n tới đích, và $h^*(n)$ là chi phí thật sự từ n tới đích. Nếu hàm đánh giá thỏa

$$h(n) \leq h^*(n),$$

thì khi đã có lời giải tốt nhất hiện tại là ans , điều kiện

$$g(n) + h(n) \geq ans$$

cho phép cắt bỏ trạng thái n .

Ràng buộc chính của bài là hai đường hầm không được cắt nhau. Nếu tạm bỏ ràng buộc này, ta thu được một bài dễ hơn: cho một tập điểm, chia chúng thành một số tập con; trong mỗi tập con, nối các điểm bằng một dây chuyền sao cho tổng chi phí nhỏ nhất. Lời giải tối ưu của bài đã nói ràng buộc là cận dưới cho chi phí còn lại của bài gốc, nên có thể dùng làm hàm đánh giá.

Gọi $f(S)$ là chi phí tối ưu cho tập điểm S , ta có quy hoạch động:

$$f(S) = \min_{T \subset S} \{f(S \setminus T) + w(T)\}.$$

Ở đây $w(T)$ là chi phí nhỏ nhất để nối các điểm trong T bằng một dây chuyền:

$$w(T) = \min_{k \in T} \{w(T \setminus \{k\}) + \text{dist}(T \setminus \{k\}, k)\}.$$

Đặt $h(S) = f(S)$, ta có $f(S) \leq h^*(S)$, nên đánh giá này hợp lệ.

Sau khi dùng hàm đánh giá, số nút còn phải xét giảm mạnh:

Độ sâu	Số nút	Độ sâu	Số nút
1	0	10	0
2	0	11	0
3	0	12	0
4	0	13	0
5	0	14	0
6	0	15	2112
7	0	16	43342
8	0	17	16733
9	0		

1.2 Ứng dụng trong một lớp bài toán khả thi

Với bài toán khả thi, mục tiêu không phải là chứng minh lời giải đang có là tối ưu, mà là nhanh chóng tìm được một cấu hình hợp lệ. Vì thế hàm đánh giá không chỉ dùng để cắt nhánh; nó còn dùng để so sánh các trạng thái con và đưa trạng thái có triển vọng lên trước.

Một ví dụ điển hình là Sokoban. Nếu tìm theo chiều đẩy hộp, chương trình phải xử lý nhiều thể kẹt: hộp vào góc, nhiều hộp chặn nhau, hộp bị ép sát tường nơi không có đích. Tìm ngược theo chiều kéo hộp làm nhiều thể kẹt biến mất một cách tự nhiên. Trạng thái có thể viết là $S(P, Q, pos)$, trong đó P là tập vị trí hộp, Q là tập ô đích và pos là vị trí người.

Nếu dùng tìm kiếm sâu lặp với giới hạn hiện tại là MAX , chi phí đã đi là g , ta cắt nhánh khi

$$g + h > MAX.$$

Một đánh giá đơn giản là tổng khoảng cách từ mỗi hộp tới một đích gần nhất:

$$h(S) = \sum_{i \in P} \min_{j \in Q} (|X_i - X_j| + |Y_i - Y_j|).$$

Đánh giá này có thể quá lạc quan vì nhiều hộp cùng chọn một đích. Cải tiến là dùng ghép cặp tối ưu giữa hộp và đích, chẳng hạn thuật toán KM trên đồ thị hai phía. Sau mỗi lần di chuyển chỉ một hộp đổi vị trí, nên có thể tận dụng kết quả ghép cặp trước đó để cập nhật nhanh hơn.

Trong loại bài này, thứ tự mở rộng trạng thái rất quan trọng. Nếu các trạng thái con được sắp theo giá trị đánh giá trước khi tìm tiếp, chương trình thường gặp lời giải hợp lệ sớm hơn nhiều.

2 Hàm đánh giá trong tối ưu quy hoạch động

USACO DEC08 GOLD – Largest Fence

Farmer John có n điểm hàng rào, với $5 \leq n \leq 250$. Ông cần dựng một vòng hàng rào là một bao lồi, và muốn số điểm nằm trên bao lồi là lớn nhất.

Một quy hoạch động trực tiếp có thể làm như sau:

1. liệt kê điểm ngoài cùng bên trái của bao lồi cuối cùng;
2. sắp các điểm còn lại theo góc cực;
3. đặt $F(i, j)$ là giá trị tốt nhất khi “bao lồi” hiện tại có hai điểm cuối là i, j ;
4. chuyển trạng thái bằng

$$F(i, j) = \max_k \{F(j, k) + 1\},$$

với điều kiện hình học bảo đảm hướng đi vẫn tạo bao lồi.

Cách này ngắn, dễ viết và tương đối ít lỗi, nhưng số trạng thái quá nhiều và mỗi trạng thái lại chuyển chậm. Độ phức tạp là $\mathcal{O}(N^4)$, không đủ cho dữ liệu lớn.

Tối ưu thứ nhất: đặc trưng đơn đỉnh

Quan sát hình học của bao lồi: nếu cố định một điểm, dãy khoảng cách tới các điểm theo thứ tự góc thường có dạng đơn đỉnh, nghĩa là tăng rồi giảm. Đặc trưng này tạo ra một cận cho số điểm còn có thể thêm.

Gọi $down(i)$ là độ dài tốt nhất của đoạn giảm bắt đầu ở i , và $up(i)$ là độ dài tốt nhất của đoạn đơn đỉnh bắt đầu ở i . Ta có

$$down(i) = \max\{down(j) + 1 \mid j > i, \text{dist}(j) < \text{dist}(i)\},$$

$$up(i) = \max(down(i), \max\{up(j) + 1 \mid j > i, \text{dist}(j) > \text{dist}(i)\}).$$

Với trạng thái $F(i, j)$, đặt

$$h(i, j) = up(j).$$

Nếu giá trị hiện tại cộng với $h(i, j)$ không thể vượt qua ANS, trạng thái đó không đáng mở rộng.

Tối ưu thứ hai: dùng đánh giá để so sánh trạng thái

Hàm đánh giá còn có thể dùng như tiêu chí sắp xếp trạng thái con. Khi các con có triển vọng được xét trước, chương trình có cơ hội tìm lời giải tốt sớm hơn; lời giải hiện tại càng tốt thì các nhánh kém càng bị cắt mạnh. Đây là cách biến cùng một công thức đánh giá thành cả công cụ cắt nhánh lẫn công cụ điều phối thứ tự tìm kiếm.

3 Hàm đánh giá trong bài tìm lời giải khá tốt

Trong một số bài, việc tìm nghiệm tối ưu quá đắt, còn yêu cầu chấm điểm hoặc ứng dụng thực tế chỉ cần nghiệm khá tốt. Khi đó hàm đánh giá có thể được dùng để chọn hướng cải thiện.

Bài toán người bán hàng là ví dụ quen thuộc. Cho n điểm trên mặt phẳng, cần tìm một chu trình đi qua mỗi điểm đúng một lần với tổng độ dài càng nhỏ càng tốt. Một trạng thái có thể là một đường đi một phần

$$Q = \{p_1, p_2, \dots, p_k\}$$

đã được chọn. Để đánh giá phần còn lại, ta nối ràng buộc chu trình Hamilton thành bài cây khung nhỏ nhất trên tập

$$(P \setminus Q) \cup \{p_1, p_k\}.$$

Chi phí MST của tập này là cận dưới cho phần đường cần hoàn tất. Khi so sánh nhiều hướng mở rộng, hướng có chi phí đã đi cộng với cận dưới nhỏ hơn sẽ được ưu tiên. Đánh giá này không bảo đảm tìm được nghiệm tối ưu trong mọi tình huống nếu ta chủ động dừng sớm, nhưng nó thường dẫn tìm kiếm về những phương án tốt hơn điều chỉnh ngẫu nhiên thuần túy.

4 Kết luận

Việc xây dựng hàm đánh giá thường bắt đầu từ việc phát hiện đặc trưng của bài toán, rồi giảm bớt một số ràng buộc để thu được bài dễ hơn. Từ bài dễ hơn, ta lấy một giá trị phản ánh triển vọng của trạng thái hiện tại.

Qua hàm đánh giá, chương trình có thể:

- đánh giá mức tốt xấu của trạng thái;
- giảm số trạng thái vô ích;
- giảm số lần chuyển trạng thái không hiệu quả;
- nâng cao đáng kể hiệu suất tổng thể.

Giới hạn của phương pháp là nó phù hợp nhất với các bài giải theo kiểu tinh chỉnh dần. Ta luôn phải cân bằng giữa chất lượng tối ưu, thời gian chạy và bộ nhớ.

Phụ lục: từ tìm kiếm tốt nhất trước tới A*

Với một trạng thái n , hàm đánh giá tổng quát thường được viết là

$$f(n) = g(n) + h(n),$$

trong đó $g(n)$ là chi phí từ trạng thái đầu tới n , còn $h(n)$ là ước lượng theo kinh nghiệm cho chi phí từ n tới đích. Tìm kiếm tốt nhất trước theo $f(n)$ dẫn tới thuật toán A. Khi $h(n)$ thỏa điều kiện không vượt quá chi phí đường ngắn nhất từ n tới đích, ta có thuật toán A*.

Một điều kiện hữu ích là tính đơn điệu. Với mọi trạng thái n_i, n_j , nếu n_j là hậu duệ trực tiếp của n_i , ta cần

$$h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j),$$

và trạng thái đích có giá trị đánh giá

$$h(\text{Goal}) = 0.$$

Lời cảm ơn

Xin cảm ơn.